

VI

CONSUMO DE SERVICIOS REST

El capítulo aborda la implementación del patrón Provider en Flutter como una estrategia moderna para la gestión eficiente del estado en aplicaciones móviles que consumen APIs REST. Inicia explicando los fundamentos de la arquitectura cliente-servidor, el funcionamiento de las APIs REST y la necesidad de mantener sincronizados los datos entre el servidor y la interfaz de usuario. Además, se describen las limitaciones del uso de “setState()” y la importancia de aplicar patrones de diseño que promuevan la modularidad y el mantenimiento del código.

Posteriormente, se desarrolla una aplicación de ejemplo llamada BooksApp, que permite realizar operaciones CRUD sobre una colección de libros alojada en un servidor remoto. A través de este caso práctico, se ilustra la estructura del proyecto, el uso de modelos de datos, servicios HTTP y la integración del patrón Provider mediante “ChangeNotifier”. Se demuestra cómo los widgets pueden reaccionar automáticamente a los cambios en los datos, logrando una interfaz fluida, coherente y libre de acoplamientos innecesarios.

Finalmente, se presentan los resultados obtenidos del análisis comparativo entre Provider y los métodos tradicionales de gestión de estado. El capítulo concluye que Provider ofrece una arquitectura más clara, escalable y mantenible, mejorando el rendimiento y reduciendo la complejidad del código.

6.1 Introducción

En el ecosistema actual del desarrollo móvil, las aplicaciones deben comunicarse constantemente con servicios web externos para obtener o enviar información. Esta interacción, generalmente implementada a través de APIs REST, plantea un desafío importante: mantener sincronizados los datos internos de la aplicación con la información que proviene del servidor.

Uno de los aspectos más complejos en Flutter y en cualquier entorno declarativo es la gestión del estado. El estado representa la información que una aplicación necesita para renderizar su interfaz de manera coherente en cada momento. Si el estado no se maneja adecuadamente, se generan errores, actualizaciones innecesarias y un código difícil de mantener.

Para abordar este problema, se ha implementado un patrón de gestión de estado ampliamente adoptado por la comunidad: Provider, el cual simplifica la comunicación entre la lógica de negocio y la interfaz gráfica, promoviendo una arquitectura más modular, mantenible y reactiva.

El objetivo de este capítulo es analizar, describir y ejemplificar la implementación del patrón Provider en una aplicación Flutter que consume datos desde una API REST, con el propósito de demostrar su eficiencia y ventajas frente a mecanismos tradicionales como setState.

6.2 Fundamentos teóricos

A continuación se abordan definiciones necesarias para lograr un adecuado entendimiento acerca del consumo de servicios REST.

6.2.1 Arquitectura cliente – servidor

La arquitectura cliente-servidor es un modelo de comunicación distribuida en el que las tareas y los recursos se dividen entre dos entidades principales:

- El cliente: es la aplicación que solicita servicios o recursos. Puede ser un navegador web, una aplicación móvil, un sistema de escritorio o incluso otro servidor. Su función principal es interactuar con el usuario y enviar peticiones al servidor.
- El servidor: es el sistema que recibe las solicitudes del cliente, las procesa y devuelve una respuesta. Generalmente, esta respuesta contiene datos obtenidos de una base de datos o el resultado de una operación lógica o computacional.

En esta relación, el cliente no necesita conocer cómo funciona internamente el servidor, y el servidor no depende de la interfaz o el entorno del cliente. Ambos pueden evolucionar de manera independiente, siempre que mantengan el mismo protocolo de comunicación.

La arquitectura cliente-servidor se puede representar como se muestra a continuación:

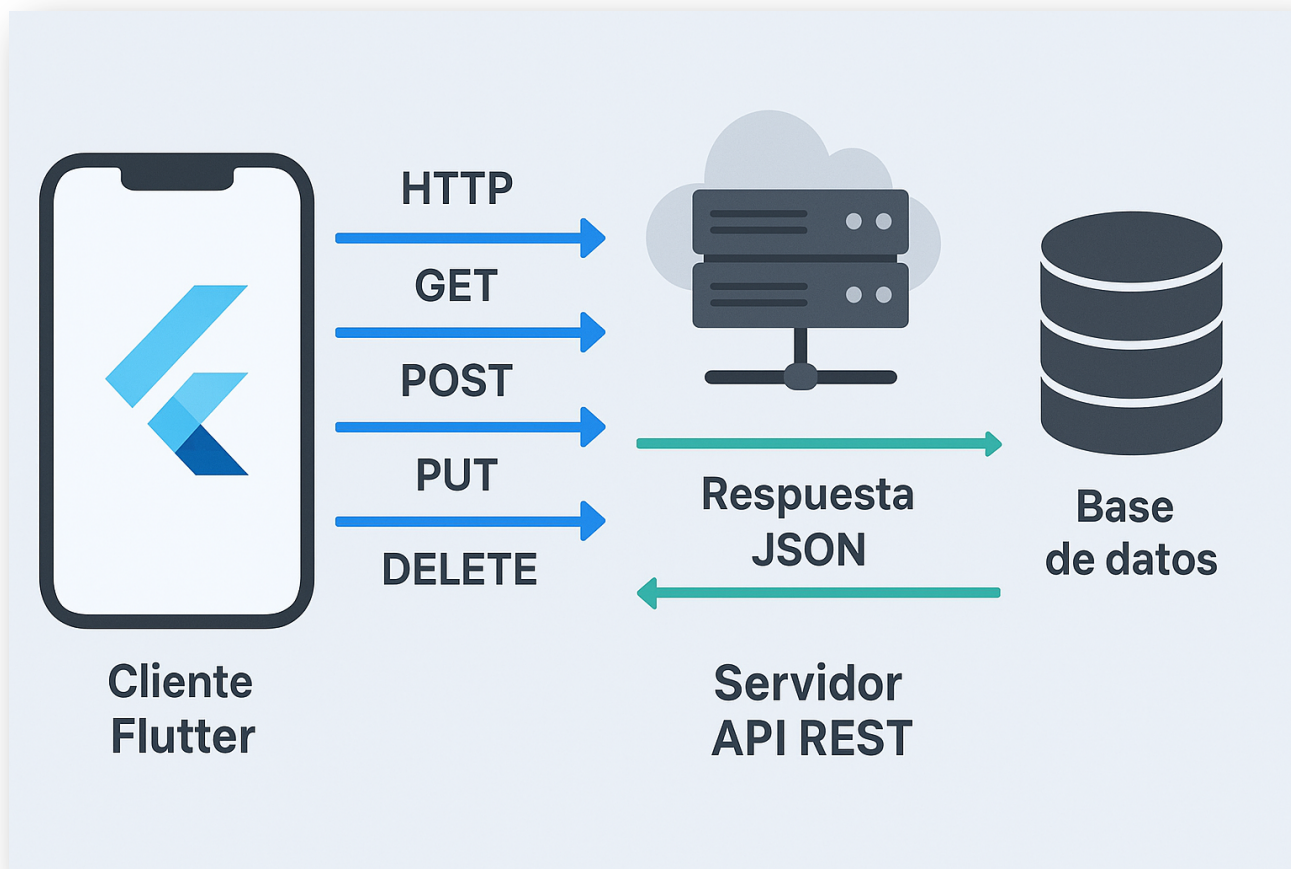


Figura VI.1. Esquema general del consumo de API REST

El proceso de interacción entre cliente y servidor puede representarse en tres pasos esenciales:

- **Solicitud (Request):** el cliente envía una petición al servidor a través de Internet. Ejemplo: una app de biblioteca solicita la lista de libros disponibles con la ruta `/api/libros`.
- **Procesamiento:** el servidor recibe la solicitud, ejecuta la lógica necesaria (por ejemplo, consulta su base de datos) y prepara una respuesta.
- **Respuesta (Response):** el servidor envía de vuelta los datos solicitados al cliente, generalmente en formato JSON o XML, para que sean interpretados y mostrados en la interfaz.

6.2.2 API REST

Una API es un conjunto de reglas que permiten que un programa se comuniquen con otro. Cuando decimos REST, nos referimos a un estilo de arquitectura que utiliza el protocolo HTTP (el mismo que usa la web) para intercambiar información en formato JSON o XML, aunque hoy en día se usa principalmente JSON.

En una API REST, el cliente (por ejemplo, una aplicación móvil o web) realiza peticiones HTTP al servidor, el cual procesa esas peticiones y devuelve una respuesta.

Los métodos más comunes son:

- GET: Obtener información
- POST: Crear un nuevo recurso
- PUT: Actualizar un recurso existente
- DELETE: Eliminar un recurso

Ejemplo:

Si una API REST de una biblioteca ofrece la ruta `/api/libros`, podrías:

- Hacer un GET a `/api/libros` para obtener todos los libros.
- Hacer un POST a `/api/libros` con datos en JSON para agregar uno nuevo.
- Hacer un GET a `/api/libros/3` para obtener el libro con ID 3.
- Hacer un DELETE a `/api/libros/3` para eliminarlo.

En resumen, una API REST es una forma estandarizada y eficiente de permitir que diferentes aplicaciones se comuniquen entre sí mediante la web, usando HTTP y JSON. Es la base de la mayoría de los servicios modernos, como redes sociales, tiendas en línea o apps móviles.

6.2.3 Ventajas del modelo cliente – servidor en REST

La combinación del modelo cliente-servidor con los principios de REST ofrece una serie de beneficios que explican su amplio uso en la actualidad:

- Separación de responsabilidades: El cliente se enfoca en la presentación y experiencia de usuario, mientras que el servidor gestiona los datos y la lógica del negocio.
- Escalabilidad: Dado que el servidor puede atender múltiples clientes (aplicaciones web, móviles, u otros sistemas), es posible ampliar la capacidad de procesamiento sin modificar el cliente.
- Independencia tecnológica: El cliente y el servidor pueden estar desarrollados en diferentes lenguajes o plataformas. Por ejemplo, un cliente en Flutter puede comunicarse con un servidor en NestJS o Django sin problemas.
- Mantenimiento simplificado: Las actualizaciones del servidor no afectan directamente al cliente, siempre que se mantenga la misma estructura de la API.

6.3 Desarrollo de la aplicación de ejemplo

Para demostrar el uso del patrón Provider en el consumo de una API REST, se desarrolló una aplicación Flutter denominada BooksApp, que permite realizar operaciones CRUD (crear, leer, actualizar y eliminar) sobre una colección de libros alojados en un servidor.

6.3.1 Estructura del proyecto

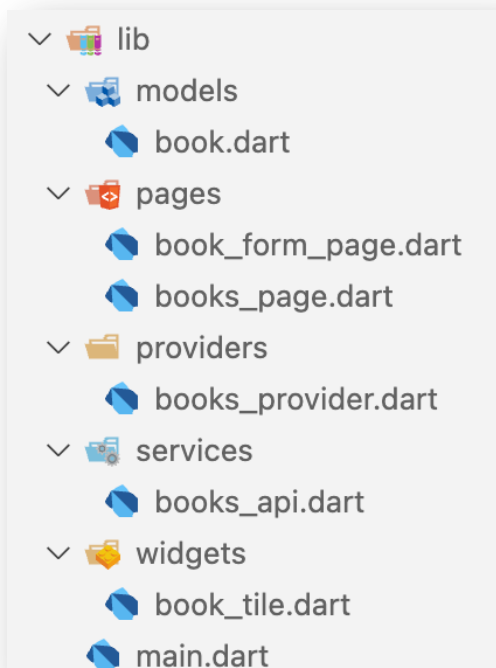


Figura VI.2. Pantallas de la APP BooksApp

Esta organización refleja la separación de responsabilidades:

- `models/`: define las estructuras de datos.
- `services/`: gestiona la comunicación con la API REST.
- `providers/`: contiene la lógica del estado global.
- `pages/`: representa las pantallas principales.
- `widgets/`: agrupa componentes visuales reutilizables.

6.3.2 Modelo de datos

La clase `Book` representa un modelo de datos para libros.

- Tiene cuatro atributos: “id”, “title”, “autor” y “year”.
- Constructor: permite crear un libro con parámetros nombrados, donde “id” es opcional.
- `fromJson()`: método de fábrica que convierte un JSON en un objeto `Book`, manejando tipos y valores nulos.
- `toJson()`: transforma el objeto `Book` a formato JSON, listo para enviarse a una API o guardarse.

En resumen, esta clase sirve para mapear datos entre una API REST y la aplicación Flutter.

book.dart

```
class Book {
  final int? id;
  String title;
  String author;
  int year;

  Book({
    this.id,
    required this.title,
    required this.author,
    required this.year,
  });

  factory Book.fromJson(Map<String, dynamic> json) => Book(
    id: json['id'] is int ? json['id'] : int.tryParse('${json['id']}'),
    title: json['titulo'] ?? '',
    author: json['autor'] ?? '',
    year: json['anioPublicacion'] is int
      ? json['anioPublicacion']
      : int.tryParse('${json['anioPublicacion']}') ?? 0,
  );

  Map<String, dynamic> toJson() => {
    if (id != null) 'id': id,
    'titulo': title,
    'autor': author,
    'anioPublicacion': year,
  };
}
```

6.3.3 Servicio de acceso a la API

En una aplicación moderna, especialmente en Flutter, el acceso a datos externos se realiza mediante API REST.

Una API REST expone recursos (como libros, usuarios o productos) a través de endpoints HTTP (GET, POST, PATCH, DELETE), permitiendo que una aplicación cliente se comunice con un servidor.

La clase BooksApi cumple precisamente esa función: conecta la app Flutter con un backend (por ejemplo, desarrollado en Node.js, Django o Laravel) para administrar los libros almacenados en una base de datos.

Para la explicación de la clase BooksApi, se hará una división del código y posteriormente se presentará el código completo.

Importaciones necesarias

```
import 'dart:convert';
import 'dart:io';
import 'package:http/http.dart' as http;
import '../models/book.dart';
```

Estas importaciones proporcionan las herramientas fundamentales:

- “dart:convert”: permite convertir datos entre JSON (texto) y objetos de Dart (jsonEncode y jsonDecode).
- “dart:io”: ofrece clases como HttpException para manejar errores en la comunicación.
- “package:http/http.dart”: es la librería estándar para enviar peticiones HTTP (get, post, patch, delete).
- “models/book.dart”: importa la clase Book, que define el modelo de datos para cada libro.

Estructura básica de la clase

```
class BooksApi {
  static const String baseUrl = 'http://localhost:3000';
```

Se define una clase llamada BooksApi con un atributo baseUrl, que indica la dirección del servidor donde se encuentra la API.

En un entorno de desarrollo, esta dirección suele apuntar a localhost, pero en producción sería algo como https://api.mibiblioteca.com.

Cliente HTTP reutilizable

```
final http.Client _client;
BooksApi({http.Client? client}) : _client = client ?? http.Client();
```

Este fragmento crea un cliente HTTP reutilizable (“_client”).

- Si no se pasa ningún cliente, se instancia uno nuevo (“http.Client”).
- Si se pasa uno (por ejemplo, en pruebas unitarias), se usa ese.

El uso de un cliente persistente es más eficiente que abrir una nueva conexión en cada petición, además de permitir control sobre su ciclo de vida.

Método privado “_u()” – Construcción de URIs

```
Uri _u(String path, [Map<String, dynamic>? query]) => Uri.parse(
  '$baseUrl$path',
).replace(queryParameters: query?.map((k, v) => MapEntry(k, '$v')));
```

Este método genera de manera limpia un objeto “Uri” combinando:

- la URL base (“baseUrl”),

- una ruta específica (“path”), como “/books” o “/books/3”,
- y, opcionalmente, parámetros de consulta (“query”), por ejemplo “?page=2&limit=10”.

Así, una llamada como:

```
_u('/books', {'page': 2, 'limit': 10});
```

producirá: `http://localhost:3000/books?page=2&limit=10`

Método “getBooks()” – Obtener todos los libros

```
Future<List<Book>> getBooks() async {
  final res = await _client.get(_u('/books'));
}
```

Este método realiza una petición GET al endpoint “/books”.

El resultado (“res”) es un objeto “http.Response” que contiene:

- el código de estado (“res.statusCode”),
- y el cuerpo (“res.body”), generalmente en formato JSON.

Verificación de la respuesta

```
if (res.statusCode >= 200 && res.statusCode < 300) {
```

Los códigos entre 200 y 299 indican éxito (por ejemplo, “200 OK” o “201 Created”).

Procesamiento del JSON

```
final decoded = jsonDecode(res.body);
```

Se convierte el cuerpo de texto a un mapa o lista de Dart.

El método admite dos formatos posibles de respuesta:

- Estructura moderna:

```
{ "data": [ ... ], "total": 10, "page": 1, "limit": 5 }
```

- Estructura simple (compatibilidad):

```
[ ... ]
```

Ambos son procesados correctamente con el siguiente código:

```
// Respuesta esperada: { data: [...], total, page, limit }
if (decoded is Map<String, dynamic>) {
  final data = decoded['data'];
  if (data is List) {
```



```
        return data
            .map((e) => Book.fromJson(e as Map<String, dynamic>))
            .toList();
    } else {
        throw const FormatException('El campo "data" no es una lista.');
```

Cada elemento de la lista se transforma en un objeto “Book” usando el método “fromJson” del modelo. Si la estructura del JSON no coincide con lo esperado, se lanza una excepción controlada.

Método “getBook()” – Obtener un libro por ID

```
Future<Book> getBook(int id) async {
    final res = await _client.get(_u('/books/$id'));
```

Este método solicita un libro específico mediante una petición GET a “/books/{id}”.

Si la respuesta es válida, el JSON recibido se convierte en un objeto “Book”.

```
    if (res.statusCode >= 200 && res.statusCode < 300) {
        final data = jsonDecode(res.body) as Map<String, dynamic>;
        return Book.fromJson(data);
    }
    throw Exception('Error al obtener el libro $id: ${res.statusCode}');
```

En caso contrario, lanza una excepción con el código de error.

Método “createBook()” – Crear un nuevo libro

```
Future<Book> createBook(Book book) async {
    final res = await _client.post(
        _u('/books'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode(book.toJson()..remove('id')),
    );
    if (res.statusCode >= 200 && res.statusCode < 300) {
        return Book.fromJson(jsonDecode(res.body));
    }
    throw Exception('Error al crear libro: ${res.statusCode}');
```

Este método envía una petición POST al servidor, agregando un nuevo libro.

- Se incluyen encabezados que indican que el contenido es JSON.
- Se envían los datos del libro convertidos con “toJson()”.
- Se elimina el `id`, ya que normalmente lo genera el servidor.

Si el servidor responde correctamente, el cuerpo devuelto se convierte nuevamente en un objeto `Book`.

Método “updateBook()” – Actualizar un libro existente

```
Future<Book> updateBook(Book book) async {
  if (book.id == null) throw Exception('ID requerido para actualizar');
  final res = await _client.patch(
    _u('/books/${book.id}'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(book.toJson()..remove('id')),
  );
  if (res.statusCode >= 200 && res.statusCode < 300) {
    return Book.fromJson(jsonDecode(res.body));
  }
  throw Exception('Error al actualizar libro ${book.id}: ${res.statusCode}');
}
```

Este método usa PATCH para actualizar parcialmente un recurso.

Primero, verifica que el libro tenga un “id”, pues sin él no se podría identificar el registro en la base de datos.

La respuesta se convierte en un nuevo objeto “Book”, reflejando los cambios aplicados en el servidor.

Método “deleteBook()” – Eliminar un libro

```
Future<void> deleteBook(int id) async {
  final res = await _client.delete(_u('/books/$id'));
  if (res.statusCode >= 200 && res.statusCode < 300) {
    return;
  }
  throw Exception('Error al eliminar libro $id: ${res.statusCode}');
}
```

Envía una petición DELETE al endpoint “/books/{id}”.

Si la respuesta es satisfactoria, simplemente termina.

De lo contrario, lanza una excepción con el código de error correspondiente.

A continuación, se presenta el código completo de la clase:

books_api.dart

```
import 'dart:convert';
import 'dart:io';
import 'package:http/http.dart' as http;
import '../models/book.dart';

class BooksApi {
  static const String baseUrl = 'http://localhost:3000';
```

```
final http.Client _client;
BooksApi({http.Client? client}) : _client = client ?? http.Client();

Uri _u(String path, [Map<String, dynamic>? query]) => Uri.parse(
  '$baseUrl$path',
).replace(queryParameters: query?.map((k, v) => MapEntry(k, '$v')));

Future<List<Book>> getBooks() async {
  final res = await _client.get(_u('/books'));

  if (res.statusCode >= 200 && res.statusCode < 300) {
    final decoded = jsonDecode(res.body);

    // Respuesta esperada: { data: [...], total, page, limit }
    if (decoded is Map<String, dynamic>) {
      final data = decoded['data'];
      if (data is List) {
        return data
          .map((e) => Book.fromJson(e as Map<String, dynamic>))
          .toList();
      } else {
        throw const FormatException('El campo "data" no es una lista.');
```

```

}

Future<Book> createBook(Book book) async {
  final res = await _client.post(
    _u('/books'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(book.toJson()..remove('id')),
  );
  if (res.statusCode >= 200 && res.statusCode < 300) {
    return Book.fromJson(jsonDecode(res.body));
  }
  throw Exception('Error al crear libro: ${res.statusCode}');
}

Future<Book> updateBook(Book book) async {
  if (book.id == null) throw Exception('ID requerido para actualizar');
  final res = await _client.patch(
    _u('/books/${book.id}'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(book.toJson()..remove('id')),
  );
  if (res.statusCode >= 200 && res.statusCode < 300) {
    return Book.fromJson(jsonDecode(res.body));
  }
  throw Exception('Error al actualizar libro ${book.id}: ${res.statusCode}');
}

Future<void> deleteBook(int id) async {
  final res = await _client.delete(_u('/books/$id'));
  if (res.statusCode >= 200 && res.statusCode < 300) {
    return;
  }
  throw Exception('Error al eliminar libro $id: ${res.statusCode}');
}
}

```

La clase “BooksApi” es un ejemplo claro de abstracción del acceso a datos.

El código Flutter que consuma esta clase no necesita saber cómo se construye la URL, cómo se maneja la codificación JSON ni cómo se interpretan los códigos HTTP.

Solo debe invocar métodos como:

- final books = await BooksApi().getBooks();
- final created = await BooksApi().createBook(Book(...));

De esta forma, la aplicación mantiene una arquitectura limpia y modular, donde la capa de datos está separada de la capa de presentación.

6.3.4 Provider para la gestión del estado

Este código define la clase BooksProvider, que actúa como intermediario entre la interfaz de usuario (UI) y la API de libros ("BooksApi").

Su función es gestionar el estado de los datos y notificar a la UI cuando estos cambian, usando el patrón Provider de Flutter, dado que el patrón ya es conocido, presentamos el código completo sin mayor detalle.

books_provider.dart

```
import 'package:flutter/foundation.dart';
import '../models/book.dart';
import '../services/books_api.dart';

enum BooksStatus { idle, loading, error }

class BooksProvider extends ChangeNotifier {
  final BooksApi api;

  BooksProvider({required this.api});

  BooksStatus status = BooksStatus.idle;
  String? errorMessage;
  List<Book> items = [];

  Future<void> fetchAll() async {
    status = BooksStatus.loading;
    errorMessage = null;
    notifyListeners();
    try {
      items = await api.getBooks();
      status = BooksStatus.idle;
    } catch (e) {
      errorMessage = e.toString();
      status = BooksStatus.error;
    }
    notifyListeners();
  }

  Future<bool> add(Book b) async {
    try {
      final created = await api.createBook(b);
      items = [created, ...items];
      notifyListeners();
      return true;
    } catch (e) {
      errorMessage = e.toString();
      notifyListeners();
    }
  }
}
```

```
        return false;
    }
}

Future<bool> update(Book b) async {
    try {
        final updated = await api.updateBook(b);
        final idx = items.indexWhere((x) => x.id == updated.id);
        if (idx != -1) {
            items[idx] = updated;
        }
        notifyListeners();
        return true;
    } catch (e) {
        errorMessage = e.toString();
        notifyListeners();
        return false;
    }
}

Future<bool> remove(int id) async {
    try {
        await api.deleteBook(id);
        items.removeWhere((x) => x.id == id);
        notifyListeners();
        return true;
    } catch (e) {
        errorMessage = e.toString();
        notifyListeners();
        return false;
    }
}
}
```

6.3.5 Inyección del Provider en la aplicación

ChangeNotifierProvider envuelve la aplicación y ofrece acceso global al BooksProvider.

main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'services/books_api.dart';
import 'providers/books_provider.dart';
import 'pages/books_page.dart';
```

```
void main() {
  runApp(const BooksApp());
}

class BooksApp extends StatelessWidget {
  const BooksApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => BooksProvider(api: BooksApi())),
      ],
      child: MaterialApp(
        title: 'Books App',
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF0066CC)),
          useMaterial3: true,
        ),
        home: const BooksPage(),
      ),
    );
  }
}
```

Cada vez que el Provider actualiza su estado, los widgets que lo observan se reconstruyen automáticamente, manteniendo sincronizada la interfaz sin intervención manual.

6.3.6 Código fuente del proyecto

Con el propósito de facilitar la comprensión práctica de los conceptos desarrollados en este capítulo, y evitar la redundancia de explicaciones técnicas ya abordadas en secciones anteriores, se pone a disposición del lector el código completo del proyecto en sus dos componentes principales: el cliente desarrollado en Flutter y el backend implementado como una API REST desarrollado en NestJS.

Estos repositorios permiten reproducir, analizar y modificar libremente los ejemplos, favoreciendo un aprendizaje activo y experimental.

- Frontend (aplicación Flutter): <https://github.com/HuascarFedor/RestApp>
- Backend (API REST NestJS): <https://github.com/HuascarFedor/booksapp>.